

Projet Deep Learning — Plateforme intelligente d'analyse d'imagerie médicale des plaies

Contexte

Un centre hospitalier souhaite se doter d'un outil d'aide au diagnostic pour l'analyse des plaies cutanées. Aujourd'hui, l'identification du type de plaie (brûlure, ulcère veineux, plaie diabétique, escarre, plaie chirurgicale, etc.) repose entièrement sur l'expertise visuelle du clinicien, ce qui entraîne :

- des **délais de diagnostic** lorsque le spécialiste n'est pas disponible ;
- une **variabilité inter-observateurs** dans la classification ;
- une **absence de mémoire institutionnelle** : les cas similaires traités par le passé ne sont pas exploités.

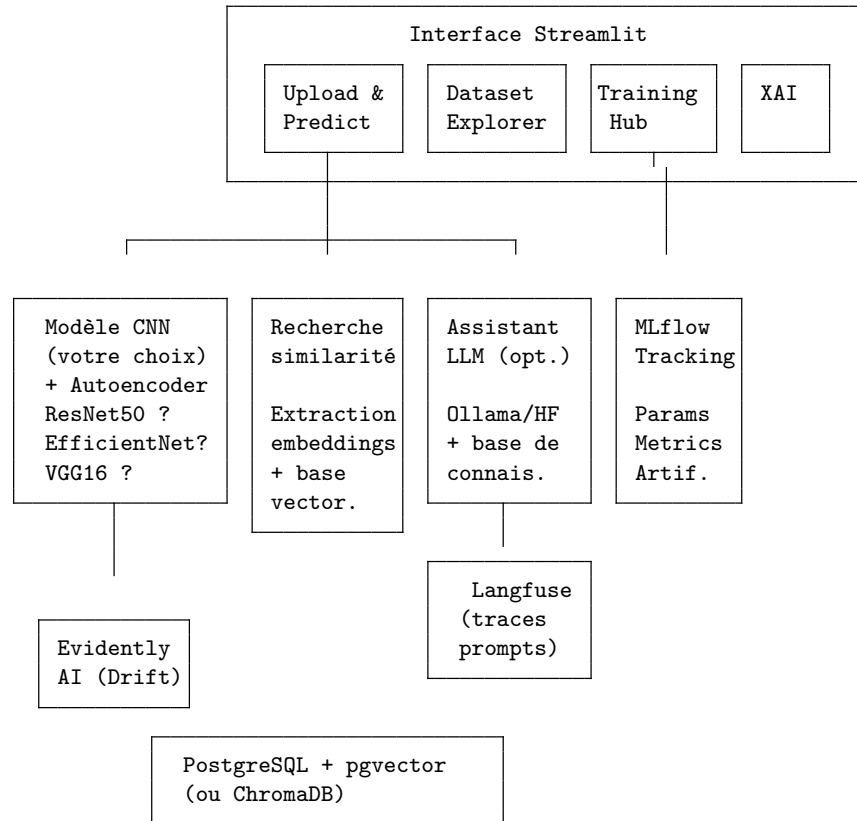
Votre mission est de concevoir une **plateforme complète** qui combine :

1. **Un modèle de Deep Learning** capable de classifier automatiquement le type de plaie à partir d'une image.
2. **Un système de recherche par similarité visuelle** permettant de retrouver des cas historiques visuellement proches dans une base de données.
3. **Un assistant IA** (optionnel mais valorisé) capable de fournir des recommandations de traitement en s'appuyant sur le diagnostic du modèle et une base de connaissances médicales.

Le projet couvre les dimensions suivantes :

Dimension	Objectif
Classification d'images (CNN)	Identifier le type de plaie parmi N classes
Autoencoder / VAE / SVDD (Deep Learning)	Détection hors-domaine (OOD)
Suivi d'expériences (MLflow)	Tracer, comparer et reproduire les entraînements
Recherche par similarité (Embeddings + base vectorielle)	Retrouver les cas historiques visuellement similaires
Explicabilité (Grad-CAM / XAI)	Visualiser les zones de l'image utilisées par le CNN
Dérive des données (Evidently AI)	Surveiller la dérive (drift) des données en production
Assistant LLM / RAG (optionnel)	Générer des recommandations de traitement contextualisées
Traçabilité LLM (Langfuse)	Tracer les prompts, réponses et latences du LLM

Architecture cible



Données

Dataset fourni

Le dataset d'images de plaies est **mis à disposition sur Moodle**. Il est organisé en sous-dossiers, chaque sous-dossier correspondant à une classe de plaie. Téléchargez-le et placez-le dans le dossier `data/raw/` de votre projet.

Important : les images ne doivent **pas** être committées dans le dépôt Git (ajoutez `data/raw/` à votre `.gitignore`). Indiquez dans le README comment télécharger le dataset depuis Moodle.

Note : Si vous considérez que le dataset est déséquilibré (certaines classes ont beaucoup plus d'images que d'autres), vous pouvez rééquilibrer.

brer via la technique de data augmentation (Exercice 1.2).

Partie 1 — Deep Learning : classification et autoencoder

Cette partie constitue le cœur du projet. Vous êtes responsables des choix d'architecture, d'hyperparamètres et de la stratégie d'entraînement. Les exercices ci-dessous définissent les livrables attendus, pas la méthode.

Exercice 1.1 — Exploration et préparation du dataset

Votre objectif : analyser le dataset choisi et le préparer pour l'entraînement.

Livrables attendus : - Analyse exploratoire (nombre d'images par classe, résolution, distribution) - Visualisation de la répartition des classes (histogramme) - Identification et traitement du déséquilibre de classes s'il existe - Pipeline de prétraitement (redimensionnement, normalisation)

Questions à traiter dans le rapport : - Votre dataset est-il équilibré ? Si non, quelles stratégies envisagez-vous ? - Quelle résolution d'entrée choisissez-vous et pourquoi ? - Comment avez-vous réparti vos données (train/validation/test) ?

Exercice 1.2 — Data augmentation et rééquilibrage des classes

Votre objectif : mettre en place une stratégie d'augmentation de données avec **PyTorch** (`torchvision.transforms`) pour : 1. Enrichir le dataset d'entraînement (augmentation classique) 2. **Rééquilibrer les classes sous-représentées** en générant davantage d'images augmentées pour les classes minoritaires

Augmentation Utilisez le framework de votre choix (ex: `torchvision.transforms` pour PyTorch ou `tf.keras.layers` pour TensorFlow) pour définir vos pipelines d'augmentation. Consultez la documentation officielle de votre framework pour la syntaxe exacte.

Stratégie de rééquilibrage Si vous considérez que le dataset est déséquilibré, plusieurs approches documentées existent pour rééquilibrer :

- **Option 1 — Échantillonnage pondéré** : suréchantillonnez les classes minoritaires pendant l'entraînement (ex: `WeightedRandomSampler` en PyTorch).
- **Option 2 — Augmentation offline** : générez des images augmentées supplémentaires pour les classes minoritaires et sauvegardez-les sur disque. Cette approche rend le rééquilibrage explicite et reproductible.
- **Option 3 — Combiner les deux** : Échantillonnage pondéré + augmentations plus agressives sur les classes minoritaires.

Éléments à considérer : - Quelles augmentations ont un sens clinique ? (rotations, retournements oui ; inversion de couleurs non) - Quel est le ratio de rééquilibrage cible ? (toutes les classes au même nombre ? un ratio intermédiaire ?) - Impact quantitatif : affichez la distribution avant/après augmentation

Questions à traiter dans le rapport : - Quelles augmentations avez-vous exclues et pourquoi ? (ex. : une inversion de couleurs a-t-elle un sens médical ?) - Quelle stratégie de rééquilibrage avez-vous choisie et quel impact a-t-elle eu sur les métriques par classe (F1 des classes minoritaires) ?

Exercice 1.3 — Entraînement du modèle CNN

Votre objectif : entraîner au moins deux architectures CNN différentes et comparer leurs performances.

Architectures suggérées (non exhaustives) : - **ResNet50** — architecture résiduelle, bonne base de comparaison - **EfficientNet (B0 à B4)** — bon compromis performance/taille - **VGG16** — architecture classique, utile comme baseline - **DenseNet**, **MobileNet**, **Inception** — selon vos ressources

Stratégies à explorer : - **Transfer Learning** : utilisation de poids pré-entraînés sur ImageNet - **Fine-tuning** : gel partiel puis dégel progressif des couches - **Entraînement from scratch** (si ressources suffisantes, à titre comparatif)

Choix d'hyperparamètres à justifier : - Optimizer (Adam, SGD + momentum, AdamW...) - Learning rate et scheduling (step decay, cosine annealing, warm-up...) - Taille de batch - Nombre d'époques et stratégie d'early stopping - Régularisation (dropout, weight decay, batch normalization)

Exercice 1.4 — Évaluation et analyse des résultats

Votre objectif : évaluer rigoureusement vos modèles et produire une analyse détaillée.

Métriques obligatoires : - **Accuracy** globale et par classe - **Matrice de confusion** (visualisée sous forme de heatmap) - **Precision, Recall, F1-Score** par classe (classification report) - **Courbes d'entraînement** (loss et accuracy sur train et validation)

Analyses attendues : - Quelles classes sont les mieux/moins bien classifiées ? Pourquoi ? - Y a-t-il des confusions récurrentes entre certaines classes ? Explication clinique possible ? - Comparaison tabulaire des architectures testées - Analyse des cas d'erreur (afficher quelques exemples mal classifiés et proposer une explication)

Question à traiter dans le rapport : - Le transfer learning apporte-t-il un gain significatif ? Sur quelles classes l'impact est-il le plus visible ?

Exercice 1.5 — Sauvegarde et chargement du modèle

Votre objectif : sauvegarder le meilleur modèle entraîné et implémenter le chargement pour l'inférence.

Éléments attendus : - Sauvegarde des poids du modèle (format `.h5`, `.pt` ou `.safetensors`) - Sauvegarde de la configuration d'entraînement (hyper-paramètres, mapping des classes) - Script ou fonction de chargement et de prédiction sur une image unique - Fonction de prédiction retournant la classe prédite, le score de confiance, et les top-3 prédictions

Exercice 1.6 — Suivi des expériences avec MLflow

Votre objectif : instrumenter vos entraînements avec **MLflow** pour tracer, comparer et reproduire vos expériences.

Concepts Lorsque l'on entraîne plusieurs modèles avec différents hyper-paramètres, il devient rapidement impossible de se souvenir de ce qui a été testé, avec quels résultats. **MLflow Tracking** résout ce problème en enregistrant automatiquement chaque *run* d'entraînement.

Entraînement ResNet50 (lr=0.001, batch=32, epochs=50)

↓
MLflow Run
├ Paramètres : lr=0.001, batch_size=32, optimizer=Adam, architecture=ResNet50
├ Métriques : val_accuracy=0.87, val_loss=0.42, f1_macro=0.83
├ Artefacts : model.h5, confusion_matrix.png, classification_report.json
└ Tags : dataset=wound_v2, augmentation=yes

MLflow UI (localhost)	
Run 1: ResNet50	acc=0.87
Run 2: EfficientNet	acc=0.91 — meilleur !
Run 3: VGG16	acc=0.79
Run 4: ResNet50*	acc=0.89
(* fine-tuné)	

Ce que vous devez logger

Catégorie	Exemples	Fonction MLflow
Paramètres	learning_rate, batch_size, optimizer, architecture, nb_epochs, dropout_rate, augmentation	<code>mlflow.log_param()</code>
Métriques	val_accuracy, val_loss, f1_macro, precision_macro, recall_macro (par epoch et finales)	<code>mlflow.log_metric()</code>
Artefacts	Poids du modèle, matrice de confusion (image), classification report (JSON), courbes d'entraînement	<code>mlflow.log_artifact()</code>
Tags	Nom du dataset, version, commentaires libres	<code>mlflow.set_tag()</code>

Indices

- Installez MLflow : `pip install mlflow`
- Lancez l'interface : `mlflow ui --port 5000` puis ouvrez `http://localhost:5000`
- Consultez la **documentation de MLflow Tracking** pour savoir comment logger des paramètres, des métriques et des artefacts : Documentation MLflow Tracking
- **Autologging** : Les fonctions d'autologging (`mlflow.tensorflow.autolog()` ou `mlflow.pytorch.autolog()`) enregistrent automatiquement les paramètres, métriques et le modèle. Vous pouvez l'utiliser comme point de départ puis personnaliser.
- Créez un **experiment** dédié par type d'expérience (ex. : `wound-classification`, `autoencoder-anomaly`).
- Organisez vos runs avec des **noms descriptifs** (`resnet50-lr0.001-aug-v2`) pour les retrouver facilement.

Livrables MLflow attendus

- Au moins **4 runs** enregistrés (2 architectures × 2 configurations d'hyperparamètres minimum)
- Capture d'écran de l'interface MLflow montrant la comparaison des runs

- Le meilleur modèle identifiable par ses métriques dans MLflow

Question à traiter dans le rapport : - Comment MLflow vous a-t-il aidé à structurer votre démarche expérimentale ? Qu'auriez-vous fait différemment sans cet outil ?

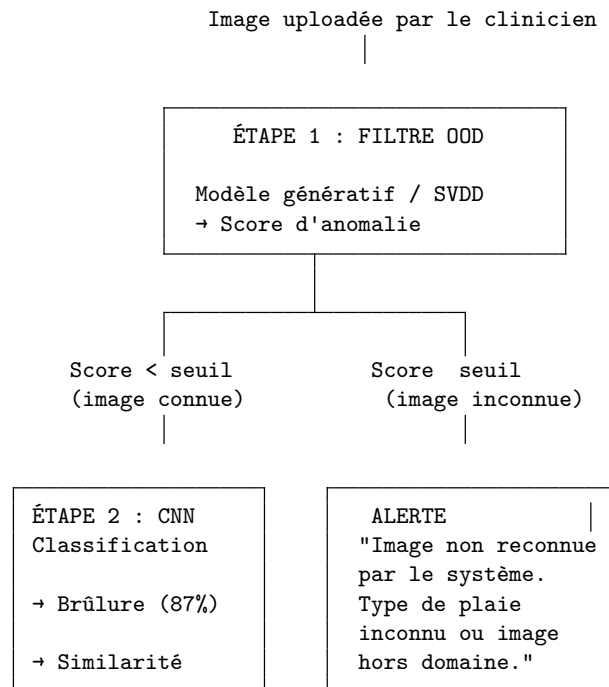
Exercice 1.7 — Modélisation de la distribution et détection hors-domaine (OOD)

Votre objectif : concevoir et entraîner un modèle non supervisé qui sert de **garde-fou** avant la classification : il détecte les images qui ne correspondent **ni** à de la peau saine, **ni** à aucune des classes de plaies connues.

Concepts — Pourquoi un filtre OOD ? Un CNN classifieur attribue **toujours** une classe à une image, même si cette image n'a rien à voir avec le domaine médical (photo de chat, paysage, radiographie...). Le score de confiance (softmax) est souvent trompeusement élevé sur des images hors domaine — le classifieur n'est pas fiable pour dire « je ne sais pas ».

Un modèle entraîné sur les images connues (saines + toutes les pathologies) apprend la distribution de son domaine. Face à une image inconnue, il produit un score d'anomalie élevé → alerte.

Pipeline de détection en deux étapes



Limites de l'Autoencoder simple et alternatives Bien qu'un Autoencoder (AE) simple utilisant l'erreur de reconstruction soit une approche classique, il présente des limites : les AEs peuvent parfois trop bien généraliser et reconstruire correctement des images OOD (raccourcis d'apprentissage). De plus, une erreur élevée de reconstruction n'implique pas toujours un cas OOD en imagerie médicale. Le choix du seuil est également critique et dépendant du dataset.

Pour un système plus robuste, vous avez le **choix de la méthode** (référez-vous à la documentation correspondante) :

1. **Autoencoder (AE) + Modélisation de l'espace latent** : Au lieu d'utiliser l'erreur de reconstruction (pixel), modélisez les embeddings latents (ex: avec un Gaussian Mixture Model - GMM) et détectez les anomalies dans l'espace latent.
2. **Variational Autoencoder (VAE)** : Ajoute une structure probabiliste offrant une meilleure estimation de l'incertitude.
3. **Deep SVDD (Support Vector Data Description)** : Apprend une région compacte des données normales. Très utilisé en détection d'anomalies moderne. Documentation : PyOD DeepSVDD.

Ce que vous devez implémenter

1. **Architecture et entraînement** :
 - Choisissez, concevez et entraînez un modèle de détection d'anomalies (AE convolutif, VAE convolutif, Deep SVDD, etc.).
 - Entraînez sur **l'ensemble du dataset** (apprentissage non supervisé).
 - Loggez l'entraînement dans MLflow.
2. **Calibration du seuil OOD** :
 - Vous êtes libres de fixer le seuil d'anomalie. Documentez-vous sur les stratégies de choix de seuil (courbes ROC/PR, percentile sur le jeu d'entraînement, etc.). Documentation utile : Scikit-Learn Outlier Detection.
 - Testez avec des **images hors domaine** (photos non médicales, autres types médicaux) pour valider.
3. **Intégration dans le pipeline** :
 - Intégrez le filtre avant le classifieur CNN.
 - Si score > seuil : retourner une alerte sans classifier.

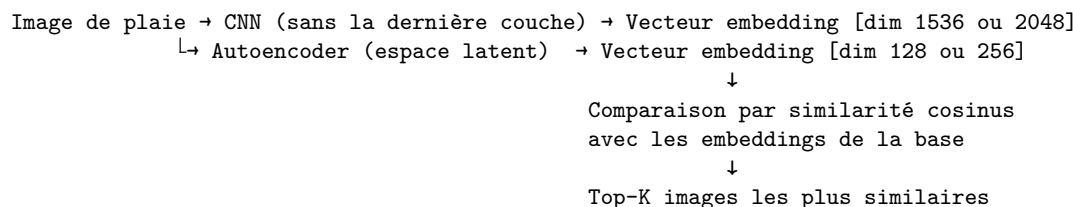
Questions à traiter dans le rapport : - Pourquoi un CNN classifieur seul ne suffit-il pas pour détecter les images hors domaine ? Donnez un exemple concret où le score de confiance (softmax) est trompeur. - Comment avez-vous choisi le seuil d'anomalie ? Quel compromis y a-t-il entre faux positifs (images connues rejetées) et faux négatifs (images inconnues non détectées) ?

Partie 2 — Recherche par similarité visuelle

Concepts

La classification vous donne un diagnostic. Mais le clinicien souhaite aussi voir des **cas similaires** traités par le passé pour comparer les traitements appliqués et leurs résultats.

La recherche par similarité repose sur les **embeddings** : des représentations vectorielles denses extraites d'une couche intermédiaire du CNN (ou de l'espace latent de l'autoencodeur). Deux images visuellement similaires produisent des vecteurs proches dans l'espace d'embedding.



Exercice 2.1 — Extraction des embeddings

Votre objectif : extraire les vecteurs d'embedding de toutes les images de votre dataset à partir de votre modèle entraîné.

Indices : - Retirez la (ou les) dernière(s) couche(s) fully-connected de votre CNN pour obtenir un extracteur de features. - Avec TensorFlow/Keras : `Model(inputs=model.input, outputs=model.layers[-N].output)`. Avec PyTorch : modifiez le `forward` ou utilisez un hook. - La dimension de l'embedding dépend de l'architecture (ResNet50 → 2048, EfficientNet-B0 → 1280, VGG16 → 4096). - Pour l'autoencodeur, l'embedding est simplement la sortie de l'encodeur (le bottleneck). - Normalisez les embeddings (L2) pour utiliser la similarité cosinus. - Stockez les embeddings avec les métadonnées associées (chemin image, classe, ID).

Exercice 2.2 — Base de données vectorielle

Votre objectif : stocker les embeddings dans une base de données vectorielle et implémenter la recherche de similarité.

Deux options sont proposées (choisissez-en une) :

Option A — PostgreSQL + pgvector (recommandé pour une architecture plus professionnelle) : - Installez PostgreSQL avec l'extension `pgvector` (via Docker recommandé) - Créez une table avec une colonne de type `vector(dimension)` - Insérez les embeddings de tout le dataset - Implémentez la requête de recherche des K plus proches voisins

Option B — ChromaDB (plus simple à mettre en place) : - Installez ChromaDB (`pip install chromadb`) - Créez une collection en mode persistant -

Insérez les embeddings avec métadonnées

Indices (Option A — pgvector) : - `docker-compose.yml` peut déclarer un service PostgreSQL avec l'image `pgvector/pgvector:pg16` - La requête de similarité cosinus avec pgvector utilise l'opérateur `<=>` : `SELECT * FROM images ORDER BY embedding <=> query_vector LIMIT k` - Pensez à créer un index IVFFlat ou HNSW pour accélérer les recherches sur de grandes bases

Exercice 2.3 — Pipeline de recherche de similarité

Votre objectif : étant donné une image de requête, trouver et afficher les K images les plus similaires de la base.

Fonctionnalité attendue : 1. L'utilisateur uploade une image 2. Le modèle CNN (ou l'autoencoder) extrait l'embedding 3. La base vectorielle retourne les K voisins les plus proches 4. L'interface affiche les images similaires avec leur score de similarité et leur classe

Partie 3 — Interface interactive (Streamlit)

Exercice 3.1 — Dashboard principal

Votre objectif : créer une application Streamlit multi-pages offrant les fonctionnalités suivantes :

Page 1 — Accueil : - Présentation du projet - Statistiques clés (nombre d'images, nombre de classes, performance du modèle)

Page 2 — Exploration du dataset : - Visualisation de la distribution des classes - Affichage d'exemples d'images par classe - Statistiques sur le dataset (résolution, répartition train/val/test)

Page 3 — Entraînement (optionnel mais valorisé) : - Choix de l'architecture et des hyperparamètres via l'interface - Lancement de l'entraînement avec affichage en temps réel des courbes de loss/accuracy - Lien vers l'interface MLflow pour consulter les runs

Page 4 — Prédiction & Analyse : - Upload d'une image de plaie - Classification automatique avec affichage du diagnostic et du score de confiance - Affichage des top-3 prédictions avec barres de probabilité - Recherche de similarité : affichage des cas historiques similaires - Score d'anomalie (autoencoder) : alerte si l'erreur de reconstruction est élevée

Page 5 — Explicabilité (XAI) (si Partie 6 implémentée) : - Affichage de la heatmap Grad-CAM superposée à l'image originale - Explication visuelle des zones utilisées par le CNN pour sa prédiction

Page 6 — Assistant IA (si Partie 5 implémentée) : - Affichage des recommandations de traitement générées par le LLM - Historique des consultations

Exercice 3.2 — Qualité de l'interface

L'interface doit être : - **Professionnelle** : design soigné, couleurs cohérentes, icônes - **Réactive** : indicateurs de chargement, gestion des erreurs - **Informative** : chaque résultat est accompagné d'explications

Partie 4 — Suivi d'expériences avec MLflow (déjà traité en Exercice 1.6)

Cette partie est intégrée dans l'Exercice 1.6 ci-dessus. Elle est rappelée ici pour clarté dans la numérotation.

L'utilisation de MLflow doit accompagner l'ensemble de vos entraînements (CNN classifieurs et autoencodeur). Consultez l'**Exercice 1.6** pour les détails d'implémentation.

Partie 5 — Assistant LLM pour les recommandations de traitement

Concepts

Le modèle de Deep Learning identifie le **type** de plaie. Mais le clinicien a besoin de recommandations de **traitement**. Un LLM (Large Language Model) peut générer ces recommandations en s'appuyant sur : - le diagnostic du CNN (classe prédite + confiance) - une base de connaissances médicales (protocoles de traitement, bonnes pratiques)

C'est le principe du **RAG (Retrieval-Augmented Generation)** : on enrichit le prompt du LLM avec des documents pertinents retrouvés dans une base de connaissances.

```
Diagnostic CNN : "Ulcère veineux" (confiance 92%)
↓
Recherche dans la base de connaissances médicales
↓
Documents retrouvés :
[1] "Protocole de traitement des ulcères veineux : compression..."
[2] "Guide de soins des plaies chroniques : nettoyage..."
↓
Prompt augmenté → LLM → Recommandation de traitement structurée
↓
Langfuse (trace du prompt, réponse, latence, tokens)
```

Exercice 5.1 — Créer la base de connaissances médicales

Votre objectif : constituer un fichier `data/base_connaissances_medicales.jsonl` contenant les protocoles de traitement pour chaque type de plaie.

Format attendu :

```
{"id": "proto-01", "type_plaie": "burns", "titre": "Protocole de traitement des brûlures", "source": "Olivier", "date": "2023-01-01"}, {"id": "proto-02", "type_plaie": "venous_ulcers", "titre": "Prise en charge des ulcères veineux", "source": "Olivier", "date": "2023-01-01"}
```

Indices : - Préparez au minimum 2-3 entrées par type de plaie (protocole de traitement, soins infirmiers, critères d'orientation vers un spécialiste). - Vous pouvez vous aider d'un LLM pour générer le contenu initial, mais **vérifiez et adaptez** les informations. - Incluez un disclaimer : ces informations sont à titre pédagogique uniquement.

Exercice 5.2 — Indexation dans une base vectorielle

Votre objectif : indexer la base de connaissances médicales dans ChromaDB (ou pgvector) pour permettre la recherche sémantique.

Indices : - Installez `sentence-transformers` pour les embeddings textuels : `pip install sentence-transformers` - Le modèle `all-MiniLM-L6-v2` (~80 Mo) est recommandé pour les embeddings textuels. - `SentenceTransformer("all-MiniLM-L6-v2").encode(liste_de_textes)` retourne un tableau NumPy. - Créez une collection dédiée (différente de celle des images) pour les documents médicaux. - Chaque document doit être indexé avec ses métadonnées (`type_plaie`, `source`).

Exercice 5.3 — Intégration du LLM

Votre objectif : intégrer un LLM local pour générer des recommandations de traitement.

Option recommandée — Ollama : 1. Installez Ollama : `ollama.ai` 2. Téléchargez un modèle : `ollama pull llama3.2` (ou `mistral`, `qwen2.5`) 3. Le serveur Ollama écoute sur `http://localhost:11434`

Indices : - L'API Ollama est simple : `POST http://localhost:11434/api/generate` avec `{"model": "llama3.2", "prompt": "..."} - En Python, la bibliothèque ollama simplifie les appels : ollama.generate(model="llama3.2", prompt="...") - Alternativement, vous pouvez utiliser un modèle HuggingFace local (Qwen2.5-1.5B-Instruct) si Ollama n'est pas disponible.`

Exercice 5.4 — Pipeline RAG pour les recommandations

Votre objectif : assembler le pipeline complet qui, à partir d'un diagnostic CNN, génère une recommandation de traitement contextualisée.

Étapes du pipeline : 1. Recevoir le diagnostic du CNN (classe prédite, confiance) 2. Rechercher les documents pertinents dans la base de connaissances (recherche

sémantique par type de plaie + par le contenu du diagnostic) 3. Construire le prompt augmenté 4. Envoyer au LLM 5. Retourner la recommandation structurée

Template de prompt suggéré :

Tu es un assistant médical spécialisé dans le traitement des plaies.

Un modèle d'intelligence artificielle a analysé une image de plaie et a produit le diagnostic suivant

Diagnostic : {classe_predite}

Confiance : {score_confiance}%

Cas similaires retrouvés : {nb_cas_similaires} cas historiques

Voici les protocoles de traitement pertinents issus de la base de connaissances :

{contexte_medical}

En te basant UNIQUEMENT sur les protocoles ci-dessus, fournis :

1. Un résumé du diagnostic
2. Les recommandations de traitement immédiates
3. Les soins de suivi recommandés
4. Les critères d'alerte nécessitant une consultation spécialisée

IMPORTANT : Ces recommandations sont fournies à titre indicatif et ne remplacent pas l'avis d'un professionnel de santé.

Exercice 5.5 — Traçabilité des appels LLM avec Langfuse

Votre objectif : tracer chaque appel au LLM avec **Langfuse** pour suivre les prompts envoyés, les réponses générées, les latences et le nombre de tokens.

Concepts Langfuse est un outil d'**observabilité LLM** open-source. Contrairement à MLflow (qui trace les entraînements de modèles), Langfuse trace les **appels au LLM en production** : quel prompt a été envoyé, quelle réponse a été générée, combien de temps cela a pris, et combien de tokens ont été consommés.

Appel au LLM

↓

Langfuse enregistre une TRACE :

- Nom : "rag-recommendation"
- Input : prompt augmenté complet
- Output : réponse du LLM
- Métadonnées : classe_predite, score_confiance, nb_docs_retrouves
- Durée : 3.2 secondes
- Tokens : input=450, output=280, total=730

Configuration Option A — Langfuse Cloud (recommandé pour démarrer)
: 1. Créez un compte gratuit sur cloud.langfuse.com 2. Créez un projet et

récupérez `LANGFUSE_PUBLIC_KEY` et `LANGFUSE_SECRET_KEY` 3. Configurez les variables d'environnement dans un fichier `.env`

Option B — Langfuse local (Docker) : - Consultez la documentation officielle pour le `docker-compose.yml` de déploiement local

Variables d'environnement (`.env`) :

```
LANGFUSE_PUBLIC_KEY=pk-lf-...
LANGFUSE_SECRET_KEY=sk-lf-...
LANGFUSE_HOST=https://cloud.langfuse.com
```

Ce fichier ne doit jamais être commité. Vérifiez que `.env` est dans votre `.gitignore`.

Indices d'implémentation

- Installez Langfuse : `pip install langfuse`
- Instanciez le client **une seule fois** au démarrage :

```
from langfuse import Langfuse
```

```
langfuse = Langfuse() # lit les variables d'environnement automatiquement
```

- Pour chaque appel au LLM, créez une **trace** :

```
trace = langfuse.trace(
    name="rag-recommendation",
    input=prompt_augmente,
    metadata={"classe_predite": classe, "confiance": score}
)
```

```
# Appel au LLM...
```

```
reponse = ollama.generate(model="llama3.2", prompt=prompt_augmente)
```

```
trace.update(output=reponse["response"])
```

```
# Enregistrer la génération (prompt + réponse + usage tokens)
```

```
trace.generation(
    name="llm-generation",
    model="llama3.2",
    input=prompt_augmente,
    output=reponse["response"],
    usage={"input": nb_tokens_input, "output": nb_tokens_output}
)
```

```
langfuse.flush() # force l'envoi des traces
```

- `langfuse.flush()` est important : Langfuse envoie les traces de manière asynchrone. Sans `flush()`, les traces risquent de ne pas être envoyées si le

programme se termine trop vite.

- Consultez le **dashboard Langfuse** pour visualiser les traces, les prompts, et les statistiques d'utilisation.

Livrables Langfuse attendus

- Au moins **5 traces** visibles dans le dashboard Langfuse
- Capture d'écran du dashboard montrant les traces avec les prompts et réponses

Exercice 5.6 — Tests du pipeline RAG

Tests minimaux à écrire :

Test	Ce qu'il vérifie
test_recherche_base	La base de données retourne des documents pertinents pour un type de plaie donné
test_prompt_augmenté	Le prompt enrichi avec le contenu LLM contient bien les documents médicaux retrouvés
test_recommandation	Le LLM retourne une réponse non vide
test_recommandation	Le LLM ne retourne pas d'avertissement médical

Partie 6 — Explicabilité visuelle / XAI (optionnel mais valorisé)

Concepts

Un CNN classe une image, mais **pourquoi** a-t-il pris cette décision ? L'**explicabilité** (XAI — eXplainable AI) permet de visualiser les zones de l'image que le réseau a utilisées pour sa prédiction. C'est essentiel dans un contexte médical : un clinicien ne fera pas confiance à un diagnostic qu'il ne peut pas comprendre.

Grad-CAM (Gradient-weighted Class Activation Mapping) est la méthode la plus utilisée. Elle produit une **heatmap** superposée à l'image originale, montrant les régions qui ont le plus contribué à la prédiction.

Image de plaie → CNN → Prédiction : "Brûlure" (92%)

↓

Grad-CAM

↓

Heatmap colorée
zone chaude =
forte contribution
zone froide =

faible contrib.

Exercice 6.1 — Implémentation de Grad-CAM

Votre objectif : implémenter Grad-CAM pour votre meilleur modèle CNN et visualiser les zones d'attention.

Principe de Grad-CAM

1. On effectue un passage forward pour obtenir la prédiction
2. On calcule les gradients de la classe prédite par rapport aux feature maps de la **dernière couche convolutive**
3. On pondère chaque feature map par la moyenne de ses gradients (importance globale)
4. On somme les feature maps pondérées et on applique ReLU $\rightarrow$ heatmap
5. On redimensionne la heatmap à la taille de l'image originale et on la superpose

Indices d'implémentation Utilisez des bibliothèques reconnues qui fournissent Grad-CAM ou d'autres solutions d'explicabilité (Grad-CAM++, Score-CAM, etc.) :

- **Pour PyTorch** : Utilisez la bibliothèque `pytorch-grad-cam`. Documentation `pytorch-grad-cam`
- **Pour TensorFlow/Keras** : Utilisez la bibliothèque `tf-keras-vis`. Documentation `tf-keras-vis`

Identifiez la **dernière couche convolutive** de votre modèle, car c'est celle qui contient la meilleure sémantique spatiale pour générer la heatmap. Pour l'intégration, vous pouvez utiliser OpenCV pour redimensionner la heatmap et la superposer à l'image originale.

Exercice 6.2 — Analyse et interprétation

Votre objectif : analyser les heatmaps Grad-CAM sur plusieurs images et produire une interprétation.

Analyses attendues : - Afficher les heatmaps pour **au moins 5 images correctement classifiées** et **3 images mal classifiées** - Le modèle regarde-t-il réellement la plaie ou d'autres éléments (arrière-plan, peau saine, artefacts) ? - Les zones d'attention sont-elles cohérentes avec l'expertise médicale ? - Comparer les heatmaps entre deux architectures différentes : regardent-elles les mêmes zones ?

Exercice 6.3 — Intégration dans Streamlit

Votre objectif : afficher la heatmap Grad-CAM dans l’interface Streamlit à côté de la prédiction.

L’utilisateur doit pouvoir : - Voir l’image originale, la heatmap, et la superposition côte à côte - Choisir la classe cible pour la heatmap (par défaut : classe prédite) - Comprendre visuellement pourquoi le modèle a fait cette prédiction

Partie 7 — Dérive des données (Data Drift) avec Evidently AI

Concepts

Lorsqu’un modèle est déployé en production (dans le service hospitalier), les données qu’il reçoit peuvent évoluer et s’éloigner de celles sur lesquelles il a été entraîné. C’est ce qu’on appelle la **dérive des données (Data Drift)**.

- **Dérive d’image** : Les nouvelles photos de plaies sont prises avec un smartphone différent, sous un éclairage différent, ou concernent un nouveau profil démographique de patients.
- **Dérive textuelle (si LLM utilisé)** : Les utilisateurs posent des questions plus longues, utilisent un nouveau vocabulaire médical, ou demandent des choses hors de portée du système RAG.

Evidently AI permet de comparer une distribution de *référence* (le dataset d’entraînement) à une distribution *courante* (les images soumises en production) pour détecter ces changements.

Exercice 7.1 — Dérive sur les embeddings (Images)

Votre objectif : utiliser Evidently AI pour surveiller la dérive des données visuelles en comparant les embeddings.

Plutôt que de comparer les pixels bruts (ce qui est inefficace), on compare les **embeddings** extraits par le CNN (ou l’autoencoder) entre le jeu de test et les images reçues en “production” (que vous allez simuler).

Indices : - Installez Evidently : `pip install evidently` - Utilisez `DataDriftPreset` sur un `DataFrame` pandas contenant les colonnes de vos embeddings (ex: `emb_1`, `emb_2`, ..., `emb_256`). - **Jeu de référence** : Un sous-ensemble aléatoire des embeddings de votre dataset d’entraînement/validation. - **Jeu de production (simulation)** : Simulez une dérive en modifiant un lot d’images (baisse forte de luminosité, ajout de bruit, ou sélection d’images hors-domaine) et extrayez leurs embeddings. - Générez le rapport au format HTML (référez-vous à la documentation d’Evidently AI pour la syntaxe exacte).

Exercice 7.2 — Dérive Textuelle (Optionnel, si Partie 5 implémentée)

Votre objectif : surveiller la dérive sur les requêtes textuelles faites au LLM (les inputs du RAG).

Indices : - Utilisez `TextOverviewPreset` d'Evidently sur les prompts utilisateurs.
- Il analysera la longueur des textes, le vocabulaire (mots hors vocabulaire), et l'analyse de sentiment. - Simulez une dérive en envoyant des requêtes très différentes (ex: requêtes non médicales, très courtes).

Exercice 7.3 — Intégration et Alertes

Votre objectif : créer un script de surveillance (monitoring) qui lit les logs de production récents et lève une alerte si un seuil de dérive est dépassé.

Indices : - Evidently peut aussi exporter au format JSON : `report.as_dict()`.
- Lisez ce JSON dans un script `drift_monitoring.py`. - Si la proportion de colonnes ayant dérivé (`share_of_drifted_columns`) dépasse un certain seuil (ex: 0.3), affichez un message d'alerte critique.

Partie 8 — Conteneurisation et déploiement (optionnel)

Exercice 8.1 — Docker Compose

Votre objectif : packager l'ensemble de la solution dans un `docker-compose.yml`.

Services suggérés : - `app` : application Streamlit - `db` : PostgreSQL + pgvector (si Option A choisie) - `ollama` : serveur LLM (si Partie 5 implémentée) - `mlflow` : serveur MLflow Tracking (optionnel)

Indices : - L'image Streamlit peut se baser sur `python:3.11-slim` - Les poids du modèle CNN ne doivent **pas** être copiés dans l'image Docker — utilisez un volume monté - Définissez les variables d'environnement dans un fichier `.env` - Le fichier `.env` ne doit **jamais** être commité (ajoutez-le à `.gitignore`)

Structure du dépôt attendue

```
projet_deep_learning_plaies/
├── data/
│   ├── raw/                # Images brutes (non committées, .gitignore)
│   ├── processed/          # Images prétraitées
│   ├── base_connaissances_medicales.jsonl  # Base médicale (Partie 5)
│   └── README.md           # Description des données et source
├── core/
│   ├── __init__.py
│   ├── config.py           # Configuration centralisée
│   └── data_processing.py  # Chargement et prétraitement des images
```

```

├── model_utils.py           # Définition et entraînement des modèles CNN
├── autoencoder.py          # Architecture et entraînement de l'autoencoder
├── grad_cam.py             # Implémentation Grad-CAM (Partie 6)
├── image_similarity.py     # Extraction d'embeddings et recherche
├── database.py             # Connexion et opérations sur la base vectorielle
├── ollama_client.py        # Client LLM (Partie 5)
├── langfuse_client.py      # Traçabilité LLM (Partie 5)
├── app-streamlit/
│   ├── Home.py            # Page d'accueil
│   ├── pages/
│   │   ├── 1_Dataset_Explorer.py
│   │   ├── 2_Training.py
│   │   ├── 3_Prediction.py
│   │   ├── 4_Explainability.py
│   │   └── 5_AI_Assistant.py
│   └── requirements.txt
├── models/                # Modèles sauvegardés (.gitignore pour les poids)
├── mlruns/                # Données MLflow (.gitignore)
├── reports/               # Rapports Evidently HTML (.gitignore)
├── notebooks/             # Jupyter notebooks d'exploration
├── scripts/
│   └── drift_monitoring.py # Script Evidently AI (Partie 7)
├── tests/
│   ├── test_data_processing.py
│   ├── test_model.py
│   ├── test_autoencoder.py
│   ├── test_similarity.py
│   ├── test_grad_cam.py
│   └── test_rag.py
├── docker-compose.yml
├── requirements.txt
├── .env
├── README.md
└── .gitignore
# Si Partie 6
# Si Partie 5
# Variables Langfuse/MLflow (JAMAIS commité !)

```

Livrables attendus

#	Livrable	Description	Critères d'évaluation
L1	Dépôt Git	Dépôt GitHub/GitLab public ou partagé	Structure correcte, commits réguliers, README clair
L2	Exploration des données	Notebook d'analyse exploratoire	Visualisations pertinentes, analyse du déséquilibre
L3	Data augmentation	Pipeline d'augmentation	Justification des choix, impact mesuré

#	Livrable	Description	Critères d'évaluation
L4	Modèles CNN entraînés	Au moins 2 architectures CNN comparées	Métriques complètes, analyse des erreurs
L5	Autoencoder	Autoencoder convolutif fonctionnel	Reconstruction visuelle, détection d'anomalies
L6	MLflow	Expériences tracées et comparées	Au moins 4 runs, comparaison dans l'UI
L7	Évaluation	Matrices de confusion, classification reports, courbes	Analyse critique et interprétation
L8	Recherche de similarité	Pipeline embedding → base vectorielle → top-K	Résultats visuels cohérents
L9	Interface Streamlit	Application multi-pages fonctionnelle	Design professionnel, UX soignée
L10	XAI / Grad-CAM <i>(optionnel)</i>	Heatmaps Grad-CAM avec analyse	Interprétation pertinente, intégration Streamlit
L11	Data Drift	Rapports Evidently AI	Simulation de dérive, alertes fonctionnelles
L12	Assistant LLM + Langfuse	Pipeline RAG fonctionnel + traces Langfuse	Recommandations contextualisées, traces visibles
L13	Docker <i>(optionnel)</i>	docker-compose.yml fonctionnel	Déploiement reproductible
L14	Rapport	Document écrit (15–20 pages)	Voir grille ci-dessous

Grille d'évaluation du rapport

Critère	Barème
Qualité et clarté de la rédaction	/2
Exploration et préparation des données (analyse, augmentation, justifications)	/2

Critère	Barème
Choix et justification des architectures CNN (transfer learning, hyperparamètres)	/3
Autoencoder : architecture, entraînement, détection d'anomalies	/3
Suivi des expériences avec MLflow (logs, comparaison, reproductibilité)	/2
Évaluation rigoureuse des modèles (métriques, matrices de confusion, analyse d'erreurs)	/3
Recherche par similarité (extraction embeddings, base vectorielle, résultats)	/2
Interface Streamlit (fonctionnalités, design, ergonomie)	/2
Analyse critique et limites (biais du dataset, erreurs du modèle, limites cliniques)	/2
Réponses aux questions de réflexion	/2
Originalité / approfondissement	/1
Dérive des données (Data Drift) avec Evidently AI	/2
Intégration LLM/RAG et Traçabilité avec Langfuse	/3
Sous-total	/29
Bonus	
XAI / Grad-CAM : heatmaps, analyse, intégration Streamlit	+2
Conteneurisation Docker fonctionnelle	+1
Total maximum	/29 + 3 bonus (ramené sur /20 ou noté sur 32)

Questions de réflexion — récapitulatif

Les réponses à ces questions doivent figurer dans le rapport écrit.

1. **Transfer learning vs entraînement from scratch** : dans le contexte de l'imagerie médicale, pourquoi le transfer learning depuis ImageNet est-il efficace alors que les images médicales sont très différentes des images naturelles ?
2. **Déséquilibre de classes** : quelles stratégies avez-vous mises en place ? Comparez au moins deux approches (oversampling, class weights, focal

loss...) et leur impact sur les métriques par classe.

3. **Autoencoder et détection d'anomalies** : quel est l'intérêt d'un autoencoder par rapport à un CNN classifieur pour détecter des images atypiques ? Un classifieur pourrait-il remplir ce rôle (indice : réfléchir au score de confiance vs à l'erreur de reconstruction) ?
4. **MLflow et reproductibilité** : comment MLflow vous a-t-il aidé à structurer votre démarche expérimentale ? Qu'auriez-vous fait différemment sans cet outil ? Quels pièges de reproductibilité avez-vous identifiés ?
5. **Embeddings visuels** : pourquoi les couches intermédiaires d'un CNN pré-entraîné produisent-elles de bons embeddings pour la similarité, même sur un domaine différent de celui d'entraînement ? Comment se comparent les embeddings du CNN classifieur vs ceux de l'autoencoder ?
6. **Similarité cosinus vs distance euclidienne** : quelle métrique avez-vous choisie pour la recherche de similarité et pourquoi ? Quels sont les avantages de chacune pour des embeddings de haute dimension ?
7. **Explicabilité (XAI) (si implémenté)** : que révèlent les heatmaps Grad-CAM sur les zones de l'image que le modèle utilise ? Le modèle se concentre-t-il sur la plaie ou sur des éléments périphériques ? Comment cela influence-t-il la confiance clinique ?
8. **Limites cliniques** : quels sont les risques d'un outil d'aide au diagnostic par IA dans le contexte médical ? Comment mitigez-vous ces risques dans votre solution ?
9. **RAG vs fine-tuning du LLM (si Partie 5 implémentée)** : pourquoi avoir choisi un RAG plutôt que de fine-tuner le LLM sur des données médicales ? Quels sont les avantages et inconvénients de chaque approche ?
10. **Data Drift (si implémenté)** : comment détecte-on une dérive sur des images (données non structurées) comparé à des données tabulaires classiques ? Pourquoi utilise-t-on les embeddings pour cela ?
11. **Langfuse et observabilité LLM (si implémenté)** : quelle est la différence entre tracer un entraînement (MLflow) et tracer un appel LLM en production (Langfuse) ? Quelles décisions opérationnelles chacun permet-il de prendre ?

Ressources

Documentation officielle

Ressource	URL
TensorFlow / Keras	https://www.tensorflow.org/api_docs
PyTorch	https://pytorch.org/docs/stable/
MLflow	https://mlflow.org/docs/latest/
Evidently AI	https://docs.evidentlyai.com/
Langfuse	https://langfuse.com/docs
Langfuse — Python SDK	https://langfuse.com/docs/sdk/python
Streamlit	https://docs.streamlit.io
pgvector	https://github.com/pgvector/pgvector
ChromaDB	https://docs.trychroma.com
sentence-transformers	https://www.sbert.net
Ollama	https://ollama.ai
pytorch-grad-cam	https://github.com/jacobgil/pytorch-grad-cam
tf-keras-vis (Grad-CAM TF)	https://github.com/keisen/tf-keras-vis
Docker Compose	https://docs.docker.com/compose/

Datasets suggérés

Dataset	Lien
Wound Classification Dataset (Kaggle)	Rechercher “wound classification” sur kaggle.com
ISIC Skin Lesion Analysis	https://www.isic-archive.com
Medetec Wound Database	http://www.medetec.co.uk/files/medetec-image-databases.html

Dépendances Python principales

```
# requirements.txt - cœur du projet
tensorflow>=2.15          # ou torch>=2.2
numpy>=1.24
pandas>=2.0
matplotlib>=3.8
seaborn>=0.13
scikit-learn>=1.4
Pillow>=10.0
opencv-python>=4.8
streamlit>=1.32
mlflow>=2.10
evidently>=0.4            # Optionnel : monitoring Data Drift
chromadb>=0.5             # ou psycopg2-binary + pgvector
sentence-transformers>=3.0

# requirements_llm.txt - si Partie 5 (LLM/RAG)
ollama>=0.3
langfuse>=2.0
```

```
python-dotenv>=1.0
```

```
# requirements_xai.txt - si Partie 6 (XAI)
```

```
grad-cam>=1.5          # PyTorch uniquement
```

```
# ou tf-keras-vis>=0.8  # TensorFlow uniquement
```

```
# requirements_tests.txt
```

```
pytest>=8.0
```

```
coverage>=7.5
```
